

Predictor-Based Cross-Episode Stitching

Using LeWM’s action-conditioned predictor to verify and discover latent-graph edges, and the case for testing synthetic HER before trusting it

Companion proposal to `main.tex`

6 September 2026

1 What this proposes, in one paragraph

The latent graph in `main.tex` identifies cross-episode edges by a single test: are two landmarks’ encoded states within a calibrated Euclidean threshold of each other. That test only ever looks at where a trajectory *already is*, so it can never propose an edge between two states that are dynamically one action apart but happen to sit at different points in latent space — a slightly different path to the same doorway, for instance. LeWM’s own action-conditioned predictor $\text{predict}(\mathbf{z}, \mathbf{a})$ is differentiable and frozen, and is exactly the tool needed to ask a stronger question than metric proximity: is there an action that actually connects these two states. The current methodology answers that with one mechanism: a small inverse-dynamics-model (IDM) ensemble that infers the connecting action for a candidate pair directly, with the frozen predictor as an independent cross-check (Section 2) — and, critically, a staged experiment ladder (Section 4) for testing it *before* trusting the riskiest downstream idea: replacing the graph’s existing, already-validated auxiliary-regression mechanism with synthetic cross-episode HER tuples built from these predicted actions. Section 3 argues that idea has a real chance of hurting performance, not just failing to help, and Section 5.1 reports the first experiment’s result.

2 Method

IDM ensemble confidence, verified against the predictor. This is the only mechanism actually in use. Train $N=5-8$ small MLP heads $\text{IDM}_\theta(\mathbf{z}_1, \mathbf{z}_2) \rightarrow \hat{a}$, supervised on real adjacent $(\mathbf{z}_t, \mathbf{z}_{t+1}, a_t)$ triples — a genuinely different model from the predictor above, since it inverts the dynamics rather than rolling them forward. Given a candidate cross-episode pair $(\mathbf{z}_i, \mathbf{z}_j)$, the ensemble infers the action each member believes would connect them; $\text{Var}(\hat{a})$ across the ensemble is a confidence signal, since independently-trained regressors should disagree more on pairs unlike anything in training. The caveat: some of that variance may reflect genuine action multimodality (several different actions produce a similar effect) rather than lack of support, so variance alone is not trusted as a confidence measure — it is combined with a second, independent check: the ensemble’s mean action is fed through the frozen predictor, and the pair is accepted only if the resulting prediction actually reproduces $\mathbf{z}_j$. Requiring agreement between two separately-trained models (the IDM, supervised end-to-end on real transitions; the predictor, pretrained on a different objective entirely) is a strictly stronger claim than either model’s self-consistency alone. Section 5.2 gives the exact construction and both verification stages (a latent-only check, then a ground-truth environment check) in full.

*Supporting latent-space diagnostics — consecutive vs. random-pair distances, a theoretical Gaussian reference, predictor error and direction-agreement, the embedding’s PCA spectrum, IDM ensemble variance across three populations, cosine similarity, and (as of a later correction) a cross-episode splice verification — are collected separately in `docs/graph-proposal/latent-diagnostics.tex`, generated by `scripts/latent_diagnostics.py`. **Important correction:** every predictor call below this point in the present document, and every predictor-error/direction number originally reported in that companion document, used a `WRONG` action invocation — a single real action zero-padded into the 10-dim slot `action_encoder` expects, compared one real step later. The checkpoint’s actual convention (`docs/leum.txt` App. D) is a frame-skip of 5: one predictor call consumes 5 concatenated real actions ($5 \times 2 = 10$ dims, no padding) and predicts 5 real steps ahead. The companion document now reports corrected numbers throughout; τ_{fwd} ’s mis-scaling below is diagnosed using the (still-valid) general argument, but its specific value and the E2 stage-1/3 experiments that followed were computed under the wrong invocation and have not been re-run correctly. What IS now validated under the corrected invocation: the IDM variance finding is unaffected (it never touched the predictor) — real transitions and the candidate pool overlap almost completely, genuinely random pairs sit an order of magnitude higher (0.0495 vs. ~ 0.005); and, most importantly, a corrected cross-episode splice test (feed the IDM’s inferred action into a real 5-action block, continue with the target episode’s own next 4 actions, predict from $\mathbf{z}_i$) shows the IDM’s action performing statistically indistinguishably from a real action on Two-Room (hybrid mean error 5.59 vs. real-block reference 5.77) — direct, positive evidence for the underlying mechanism that the untested "candidate for a fix" framing below could not yet claim.*

3 The central risk: synthetic HER is not free

Standard same-episode hindsight relabeling is, in a precise sense, free training data: the relabeled goal is defined as whatever state the trajectory actually reached, so the label is trivially true by construction and carries no noise. The mechanism in Section 2 that identifies a *cross-episode* connection breaks that property the moment its output — an inferred action — is treated as a real transition. If the inference is wrong, the resulting (s, a, s', g) tuple is a genuine falsehood about the environment, not a relabeling of something that happened.

That distinction matters most for where the tuple is used. Feeding a graph-derived distance into the existing auxiliary regression term on $V(s, g)$ (`main.tex`’s mode 2 consumption algorithm) injects a soft, scalar bias that is still tempered by V ’s own TD/expectile regression against real Bellman targets — and even that soft channel has already been shown, on Push-T’s noisy tiers, to invert the actor’s data preference under the wrong scale (see `main.tex`, Push-T section). Feeding a synthetic action directly into the actor’s imitation loss, $(\hat{a} - a_{\text{pred}})^2$, has no such anchor: a wrong target is a wrong target, with nothing pulling the policy back toward grounded behavior.

There is a direct precedent in this project for why a mechanism that looks safe on paper can still fail in practice: potential-based reward shaping (`main.tex`’s mode 3) is *provably* policy-invariant — a strictly stronger theoretical safety guarantee than anything available for synthetic HER — and it still lost on 4 of 5 Two-Room tiers, because TD compounds the graph’s noise even though the shaping itself cannot change the optimal policy in the exact case. Synthetic cross-episode HER has a weaker guarantee than mode 3 had. There is therefore no basis to assume the gap between a baseline and a synthetic-HER-augmented policy stays non-negative, and every reason to expect the false-edge count — and thus the risk — to grow, not shrink, as the graph gets larger and the

number of candidate cross-episode pairs grows with it. Section 4 exists to measure this rather than assume an answer either way.

4 Experiment ladder

Six gated stages, Two-Room first (a trustworthy oracle distance exists to diagnose against), Push-T only once the mechanism is trusted — the same sequencing `main.tex` used for its own B0–B5 ladder. Each stage’s kill criterion is checked before the next stage is attempted.

Table 1: Experiment ladder for predictor-based cross-episode stitching.

	Question	Method	Kill criterion / status
E1	Can an IDM ensemble recover a known action at all?	Train on real adjacent Two-Room transitions; test on held-out same-episode pairs (Algorithm 1 sketch).	Done — passed . 98.4% MSE reduction vs. marginal baseline (Section 5.1).
E2	Does IDM-ensemble variance track true confidence, and does a wide kNN candidate pool, filtered by ensemble variance and verified against the frozen predictor, beat plain kNN on precision?	Wide top-20 kNN candidates on Two-Room’s B0 landmark set; IDM ensemble infers the connecting action per pair; accept iff variance is typical AND the predictor’s forward step reproduces the target.	Open . The absolute-floor criterion FAILED to discriminate; ground-truth env stepping shows the IDM’s inferred actions are genuinely correct, even cross-trajectory, so the mechanism itself is sound (Section 5.2). A figure-motivated fix (recalibrated variance + direction agreement) was tried next and made precision <i>worse</i> ($0.717 \rightarrow 0.609$), for a diagnosed, scale-dependent reason — no working latent-only criterion exists yet. E3 remains blocked on this.
E3	Does graph quality from these edges improve, or degrade, with scale?	Re-run the B0 Spearman-vs-oracle gate at 50/300/1000-episode landmark counts with E2’s edges added.	Not yet run. Kill criterion: no improvement, or degradation at larger scale, answers the scaling question directly.

(continued on next page)

(Experiment ladder, continued)

	Question	Method	Kill criterion / status
E4	Design A — do verified edges help when fed only to the existing (safe) aux-regression mechanism, no synthetic actor targets?	Wire E3’s edges into ϕ_{dist} as ordinary identification edges; sweep tiers $\times 3$ seeds.	Not yet run. Kill criterion: if this doesn’t help, Design B is even less likely to.
E5	Design B — does synthetic cross-episode HER, fed into actor_loss, ever beat baseline, and does the gap survive scale?	Sweep a confidence-gated synthetic-tuple fraction $\in \{0, 0.05, 0.15, 0.30, 1.0\}$ across tier sizes.	Not yet run. This is the concrete test of Section 3’s concern: a negative gap at any fraction/scale confirms it.
E6	Does the winning Two-Room configuration transfer to Push-T?	Repeat E4/E5’s best config under <code>eval_protocol=same_episode</code> ; watch for the known expert-vs-noisy AWR weight-inversion failure mode.	Not attempted before Two-Room shows a clear non-negative signal.

5 Results

5.1 E1: IDM ensemble sanity check

Algorithm 1 E1 methodology

- 1: **Data:** Two-Room’s existing B0 landmark cache, 300 episodes $\rightarrow$ 27,675 frames
 - 2: **Pairs:** all real adjacent (transition) pairs, 27,375 total
 - 3: **Split:** 80/20 by *episode* (240 train / 60 test episodes $\rightarrow$ 21,910 / 5,465 pairs) — same convention as the B3 tier-setup split
 - 4: **Model:** 6 independent MLPs (hidden 256), each on its own bootstrap resample and weight init, regressing a_i from $\text{concat}(\mathbf{z}_i, \mathbf{z}_j)$, 3000 Adam steps
 - 5: **Baseline:** predict the training-set mean action for every test pair (the “marginal action distribution”)
 - 6: **Kill criterion:** held-out ensemble MSE must be meaningfully below the marginal baseline’s MSE
-

Results.

Ensemble variance on these (in-distribution) held-out pairs: mean 0.0051, std 0.0075.

Takeaways.

- The ensemble reliably inverts Two-Room’s one-step dynamics on pairs shaped like its training distribution — real, same-episode, one step apart. The kill criterion (a clear improvement over predicting the mean action) passes by a wide margin, not a marginal one.

Table 2: E1 result. MSE on held-out same-episode transitions, 60 held-out episodes never seen in training.

	MSE
marginal-action baseline	0.7544
per-member average	0.0170
ensemble mean	0.0120
improvement over baseline	98.4%

- This says nothing yet about genuinely cross-episode pairs, which is what E2 tests. Variance being small and reasonably stable here is the necessary baseline for E2 to compare against, not evidence that variance will behave the same way on out-of-distribution pairs.
- This clears the door to E2. It does not, on its own, provide any evidence about Section 3’s actual concern — that is a claim about E5, five stages away.

5.2 E2: kNN candidates, IDM variance filter, predictor verification

Rather than borrowing an unrelated real action from a neighboring state (the earlier, abandoned design described in Section 2), have the IDM ensemble infer the action connecting a SPECIFIC candidate pair $(\mathbf{z}_i, \mathbf{z}_j)$ directly, and accept the pair only if two independently-trained models agree: the ensemble’s own variance is no worse than typical, *and* feeding its mean action through the frozen predictor reproduces $\mathbf{z}_j$. Tested in two stages — a cheap latent-only check first, then a ground-truth check in the real environment on whatever the latent check couldn’t settle.

Algorithm 2 E2 stage 1: candidate construction and latent-space verification

- 1: **Candidates:** unfiltered top-20 nearest cross-episode neighbors by raw latent distance (`raw_topk_cross_episode_pairs`), NOT the calibrated $k=4$ identification-edge threshold — 298,983 pairs, chosen deliberately looser so the verification step has a real mix of true/false candidates to discriminate, unlike testing it on an already-92.7%-precision set
 - 2: **IDM ensemble:** 6 MLPs, same architecture and training split as E1, trained on real adjacent transitions only
 - 3: **for** each candidate pair $(\mathbf{z}_i, \mathbf{z}_j)$ **do**
 - 4: $\{\hat{a}_1, \dots, \hat{a}_6\} \leftarrow \{\text{IDM}_k(\mathbf{z}_i, \mathbf{z}_j)\}$; $\bar{a} \leftarrow$ their mean; $\text{Var}(\hat{a}) \leftarrow$ their variance
 - 5: **require** $\text{Var}(\hat{a}) < \tau_{\text{var}} \quad \triangleright \tau_{\text{var}} = 90\text{th percentile of the ensemble’s OWN variance on real held-out adjacent pairs — “no worse than typical”}$
 - 6: $\hat{\mathbf{z}} \leftarrow \text{predict}(\mathbf{z}_i, \text{enc}_a(\bar{a}))$
 - 7: **require** $\|\hat{\mathbf{z}} - \mathbf{z}_j\|^2 < \tau_{\text{fwd}} \quad \triangleright \tau_{\text{fwd}} = 90\text{th-percentile one-step predictor error on real transitions, 51.0}$
 - 8: **accept** iff both requirements hold
 - 9: **end for**
-

Stage 1 result: does not discriminate. 87.8% of all candidates pass both checks; precision (oracle distance ≤ 2) moves from 0.711 unfiltered to only 0.719 filtered; recall on pairs already known to be good (0.876) is statistically indistinguishable from the overall pass rate (0.878) — the filter accepts or rejects almost independently of correctness.

Diagnosis. Candidate pairs sit only 2.74 apart in raw latent distance on average; a real one-step transition moves 11.76 on average — a $4.3\times$ gap. τ_{fwd} was calibrated at the larger (real-transition) scale, so a candidate pair already well inside it before any action is applied passes almost automatically. Confirmed directly: the IDM’s inferred mean-action norm on these candidates is 0.42 versus 1.20 for real training actions — the ensemble is agreeing on a near-zero, near-identity “explanation” that trivially satisfies both checks without asserting anything dynamically real. This could be a fixable calibration-scale mismatch (e.g. a RELATIVE-improvement criterion instead of an absolute floor borrowed from a different scale) rather than evidence the two-model idea is unsound — Stage 2 settles which.

Algorithm 3 E2 stage 2: ground-truth environment verification

Require: candidate (i, j) ; its inferred mean action $\bar{a}$ from stage 1

- 1: reset the real Two-Room environment at state_i , with $\text{options} = \{\text{state} : \text{state}_i, \text{target_state} : \text{state}_j\}$
 - 2: force speed $\leftarrow 5.0$ $\triangleright$ the dataset’s empirically fixed value — mean 4.989, std 0.147, capped at exactly 5.0 over 200 episodes’ real unclamped transitions — overriding `reset()`’s otherwise-randomized per-episode draw, which would otherwise confound the replay
 - 3: step the environment with $\bar{a}$; read the REAL resulting position state'_i
 - 4: compare $\|\text{state}'_i - \text{state}_j\|$ against: (a) $\|\text{state}_i - \text{state}_j\|$ (a no-op control), and (b) the same step under a random action of equal magnitude (a control for “any small nudge helps”)
-

Stage 2, result 1 (all top-20 candidates): the action is real, not degenerate. Mean real distance $1.66 \rightarrow 0.35$ after the action (79% reduction), beating both the no-op control (1.66, identical by construction) and the random-direction control (2.30, worse than the no-op, as expected for pure noise) on 95.4% / 95.6% of pairs respectively. But these top-20 pairs are overwhelmingly near-duplicates in real position (only 0.8% of the whole 298,983-pair pool sit more than 10 apart), so this alone only shows local correction between already-similar states.

Stage 2, result 2 (restricted to the genuinely far-apart 4.2% of the pool, real distance ≥ 5): real cross-trajectory stitching. Mean inferred-action norm here is 1.14 — essentially matching real training actions’ 1.20, not a degenerate near-zero action, because the task genuinely requires real movement this time. Median real distance $6.87 \rightarrow 1.69$ (75% reduction) after the action; 95.6% of pairs improve over doing nothing, 91.8% improve over the random-direction control. The environment’s own success threshold (distance < 16.0) barely moves ($0.784 \rightarrow 0.796$), because many of these pairs are simply farther apart than one action (bounded by speed = 5) can close in a single step regardless of correctness — the median-distance reduction and the beat-the-random-control margin are the informative numbers, not the raw hit rate against a threshold tuned for full-episode navigation.

Takeaways.

- Stage 2 revises stage 1’s verdict. The IDM’s inference mechanism — ask it directly what action connects a specific candidate pair — is genuinely good, confirmed in the real environment for pairs that are actually far apart, not merely near-duplicates. What failed in stage 1 was its LATENT acceptance criterion (an absolute floor calibrated at the wrong length scale), not the underlying inference.
- Practical implication: do not run real-environment stepping as the production verification mech-

anism at scale or for a new environment without a clean resettable oracle (Push-T) — that defeats the point of a learned latent-space graph, and Two-Room’s fast grid oracle is a lucky exception, not the general case. Use stage 2’s ground truth as a CALIBRATION set instead: fix stage 1’s acceptance criterion (the relative-improvement fix proposed above) against what real stepping confirms here, then trust the resulting cheap, latent-only proxy for construction at scale.

- This reopens the door to E3 onward, on a corrected latent criterion — attempted next, in stage 3.

Stage 3: the corrected criterion, tried and failed differently. Two fixes were implemented, motivated directly by `latent-diagnostics.tex`’s figures: the variance threshold recalibrated as the geometric mean of the real-transition and random-pair variance medians (Figure 6: real 0.00505, candidate pool 0.00560, random 0.0495), and the absolute forward-error floor replaced with direction-of-displacement agreement, $\cos(\text{predict}(\mathbf{z}_i, \bar{a}) - \mathbf{z}_i, \mathbf{z}_j - \mathbf{z}_i) > 0.5$ (Figure 4’s median 0.924 on real transitions). Re-run on the same candidate pool, this made things *worse*, not better: precision $0.717 \rightarrow 0.609$, recall on known-good pairs only 46.5%.

A distance-bucket breakdown pins down why:

Table 3: Candidates bucketed by raw latent distance $\|\mathbf{z}_i - \mathbf{z}_j\|$ (3000-pair oracle-scored sample).

raw distance	frac. actually good	mean $\cos_{\text{dir}}$	recall (good kept)	precision among kept
[0, 1)	0.994	0.144	0.292	1.000
[1, 2)	0.947	0.292	0.377	0.950
[2, 3)	0.777	0.395	0.463	0.780
[3, 5)	0.489	0.635	0.691	0.445
[5, 10)	0.077	0.805	1.000	0.085

Two distinct failures, at opposite ends of the scale. Near-duplicate pairs (the bulk of the pool, the first two rows) are 95–99% genuinely good, but a target displacement this tiny has a direction dominated by encoder noise, not real geometry — mean cosine is only 0.14–0.29 for pairs that are almost all correct, so the filter wrongly rejects most of the pool’s best pairs. This is the same scale-mismatch failure as the original absolute floor, relocated to a different statistic rather than fixed. At larger gaps (last row), only 7.7% are actually good yet mean cosine is 0.805 and everything kept is 91.5% wrong — the predictor’s displacement direction does not track the specific target at this scale, it looks like a generic directional bias rather than genuine confirmation.

This is not a threshold-tuning problem. No single global cosine threshold fixes both failure modes at once: the small-gap failure needs a lower bar (or no direction check at all), while the large-gap failure needs a direction signal that is target-specific, which a single dot product can’t provide regardless of where the bar is set. A working fix would need to be scale-aware — e.g. skip the model-based check entirely below some raw distance, since near-duplicates are already well-served by the existing metric threshold alone — and would still need to solve the large-gap problem separately; neither is attempted here.

6 Status

E1 complete and passed. E2 (wide kNN candidates, IDM variance filter, predictor verification) is **not complete** after three attempts (Section 5.2): stage 1’s original absolute-floor criterion

failed to discriminate; stage 2’s ground-truth environment stepping confirmed the underlying mechanism — the IDM directly inferring the connecting action for a candidate pair — is genuinely correct, including for real cross-trajectory pairs; but stage 3’s figure-motivated fix (recalibrated variance threshold plus direction-of-displacement agreement in place of the absolute floor) made precision *worse* ($0.717 \rightarrow 0.609$), for a diagnosed, scale-dependent reason (direction is undefined for near-duplicates and non-specific at larger gaps) rather than a tuning mistake. **Net effect:** the mechanism is validated, but no working latent-only acceptance criterion exists yet — E3 onward remains blocked until one is found, most likely a scale-aware criterion that treats near-duplicate and larger-gap candidates differently rather than a single global threshold of any kind. Full artifacts: `scripts/idm_gate.py`, `scripts/common/idm_ensemble.py`, `config/graph/idm_gate.yaml`, `scripts/cycle_consistency_gate.py`, `config/graph/cycle_consistency_gate.yaml`, `scripts/env_verify.py`, results at `outputs/idm_gate_tworoom/e1_idm_gate_results.json`, `outputs/idm_gate_tworoom/e2b_cycle_consistency_gate_results.json` and `outputs/idm_gate_tworoom/e2c_env_verify_results.json`. The staged plan this document formalizes is also tracked as Markdown at `docs/experiment-plan-predictor-stitching.md`.